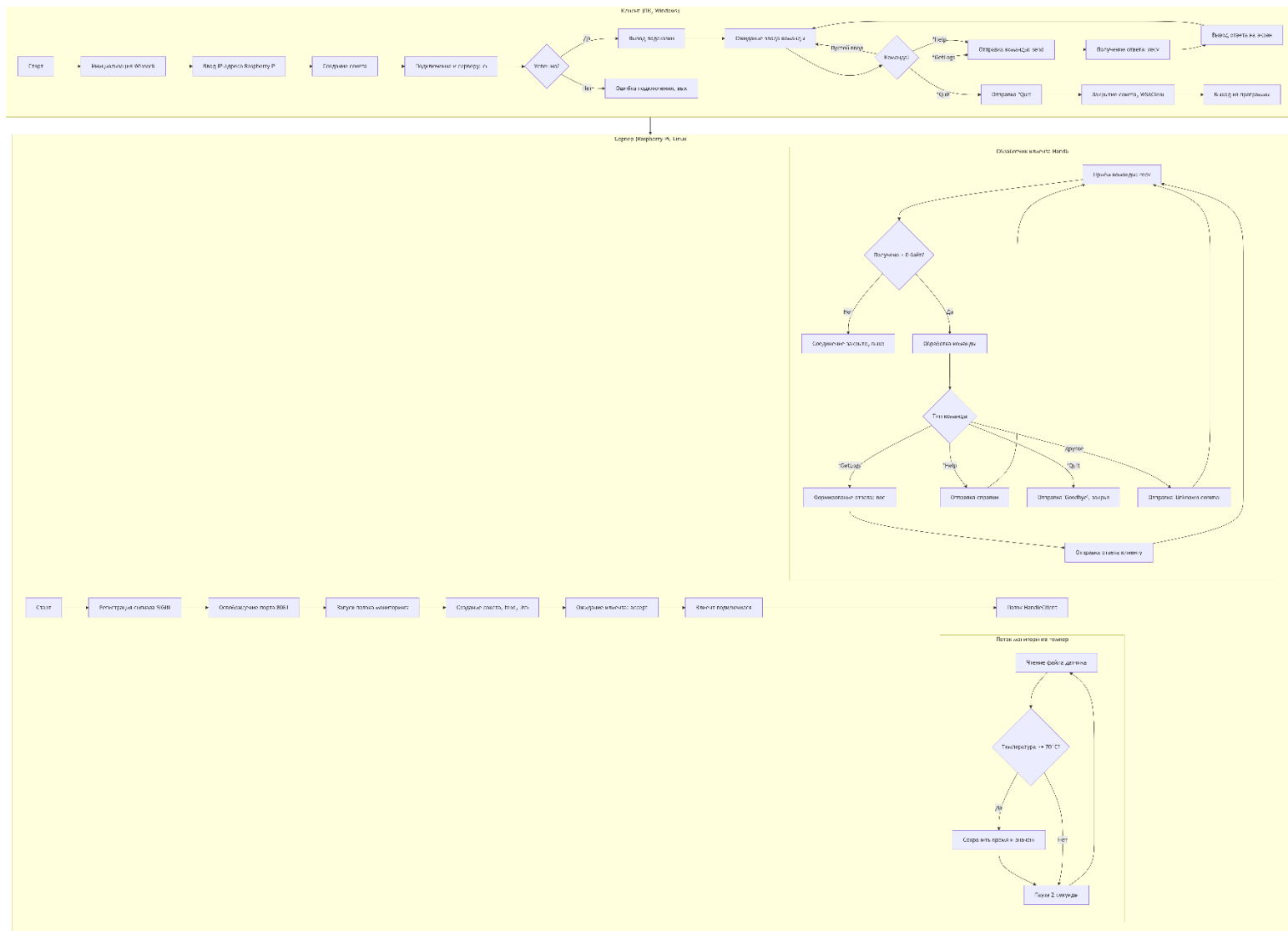




Клиент-серверный монитор температуры DS18B20

Проект состоит из двух программ:

- **Сервер** (Raspberry Pi) постоянно измеряет температуру цифровым датчиком DS18B20, подключённым через 1-Wire, фиксирует моменты превышения порога (30 °C) и ожидает TCP-подключений на порту 8082. По команде `*GetLogs` отдаёт отчёт о превышениях с момента последнего запроса и текущую температуру.
- **Клиент** (Windows) подключается к серверу по IP, отправляет текстовые команды и печатает ответы.

Архитектура сервера

Потоки

Сервер многопоточный:

1. **Главный поток** – инициализация и `accept()`.

2. **Мониторинг температуры** – отдельный `std::thread` (`TemperatureMonitor`).
3. **Обработчики клиентов** – при каждом входящем соединении создаётся `std::thread` (`HandleClient`), который сразу `detach()`.

```
std::thread monitor_thread(TemperatureMonitor);  
...  
while (g_server_running) {  
    int client_socket = accept(...);  
    std::thread client_thread(HandleClient, client_socket);  
    client_thread.detach();  
}
```

Поиск датчика

При запуске сервер автоматически находит первый датчик DS18B20 в `/sys/bus/w1/devices/` (папка с префиксом 28-):

```
std::string find_ds18b20() {  
    DIR* dir = opendir("/sys/bus/w1/devices/");  
    if (!dir) return "";  
    struct dirent* entry;  
    while ((entry = readdir(dir)) != nullptr) {  
        std::string name = entry->d_name;  
        if (name.find("28-") == 0) {  
            closedir(dir);  
            return "/sys/bus/w1/devices/" + name + "/w1_slave";  
        }  
    }  
    closedir(dir);  
    return "";  
}
```

Результат сохраняется в глобальную переменную `g_sensor_file`.

Чтение температуры

Температура получается из псевдофайла `w1_slave`:

```
float ReadDS18B20Temperature() {  
    std::ifstream file(g_sensor_file);  
    if (!file.is_open()) return -1.0f;  
  
    std::string line;  
    if (!std::getline(file, line)) return -1.0f;  
    if (line.find("YES") == std::string::npos) {
```

```

        // CRC ошибка
        return -1.0f;
    }
    if (!std::getline(file, line)) return -1.0f;
    size_t pos = line.find("t=");
    if (pos == std::string::npos) return -1.0f;
    int raw = std::stoi(line.substr(pos + 2));
    return raw / 1000.0f;    // перевод в °C
}

```

Мониторинг температуры

В цикле (интервал `kMonitorIntervalSec = 2` секунды) вызывается функция чтения. При значении $\geq$ `kCriticalTemp` (30°C) создаётся запись в общем векторе:

```

void TemperatureMonitor() {
    while (g_monitor_running) {
        float temp = ReadDS18B20Temperature();
        if (temp >= kCriticalTemp) {
            std::lock_guard<std::mutex> lock(g_data_mutex);
            g_overheat_events.push_back({std::time(nullptr), temp});
        }
        std::this_thread::sleep_for(std::chrono::seconds(kMonitorIntervalSec));
    }
}

```

Обработка команды `*GetLogs`

При поступлении команды сервер захватывает мьютекс, формирует отчёт только по событиям, случившимся после `g_last_request_time`, и обновляет это время:

```

if (request == "*GetLogs") {
    std::lock_guard<std::mutex> lock(g_data_mutex);

    std::ostringstream oss;
    if (g_last_request_time == 0)
        oss << "This is your first request.\n";
    else
        oss << "Last request time: " << format_time(g_last_request_time) << "\n";

    std::vector<OverheatEvent> recent;
    for (const auto& ev : g_overheat_events)
        if (ev.timestamp > g_last_request_time)

```

```

        recent.push_back(ev);

    oss << "\nTemperature excesses since last request:\n"
        << format_overheat_intervals(recent);
    oss << "Current DS18B20 temperature: " << ReadDS18B20Temperature() << "°C\n";

    g_last_request_time = std::time(nullptr);
    response = oss.str();
}

```

Группировка интервалов

Последовательные события с промежутком $\leq kMonitorIntervalSec + 1$ (3 с) склеиваются в один интервал:

```

std::string format_overheat_intervals(const std::vector<OverheatEvent>& events) {
    std::ostringstream oss;
    size_t i = 0;
    while (i < events.size()) {
        std::time_t start = events[i].timestamp;
        std::time_t end = start;
        float max_temp = events[i].temperature;
        size_t j = i + 1;
        while (j < events.size() &&
            difftime(events[j].timestamp, events[j-1].timestamp) <= 3) {
            end = events[j].timestamp;
            if (events[j].temperature > max_temp) max_temp = events[j].temperature;
            ++j;
        }
        if (start == end)
            oss << " - At " << format_time(start) << " : " << max_temp << "°C\n";
        else
            oss << " - From " << format_time(start) << " to " << format_time(end)
                << " : max " << max_temp << "°C\n";
        i = j;
    }
    return oss.str();
}

```

Обработка команд *help и *quit

- *help – возвращает список команд.
- *quit – отправляет "Goodbye.\n" и выставляет флаг завершения цикла обработки клиента.

```

else if (request == "*Quit") {
    response = "Goodbye.\n";
    send(client_socket, response.c_str(), response.size(), 0);
    client_connected = false;
}

```

Завершение сервера по Ctrl+C

Настроен обработчик `SIGINT`. Он сбрасывает глобальные флаги и закрывает слушающий сокет, благодаря чему `accept()` выходит с ошибкой и главный поток завершается.

```

void SignalHandler(int signal) {
    if (signal == SIGINT) {
        g_server_running = false;
        g_monitor_running = false;
        if (g_server_socket != -1) close(g_server_socket);
    }
}

std::signal(SIGINT, SignalHandler);

```

Перед стартом сервер освобождает порт 8082, чтобы гарантировать запуск:

```
system("fuser -k 8082/tcp > /dev/null 2>&1");
```

Клиент однопоточный, использует WinSock2. Подключается к серверу и в цикле отправляет команды.

Инициализация и подключение

```

WSADATA wsa_data;
WSAStartup(MAKEWORD(2,2), &wsa_data);
SOCKET sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

sockaddr_in addr{};
addr.sin_family = AF_INET;
addr.sin_port = htons(8082);
addr.sin_addr.s_addr = inet_addr("10.42.0.1"); // или ввод пользователя
connect(sock, (sockaddr*)&addr, sizeof(addr));

```

Отправка команды и приём ответа

```
std::string cmd = "*GetLogs";
```

```
send(sock, cmd.c_str(), cmd.length(), 0);
```

```
char buf[4096] = {};
```

```
int n = recv(sock, buf, sizeof(buf)-1, 0);
```

```
std::string response(buf, n);
```

```
std::cout << response;
```

Цикл работы

```
while (true) {
```

```
    std::cout << "> ";
```

```
    std::string cmd;
```

```
    std::getline(std::cin, cmd);
```

```
    if (cmd.empty()) continue;
```

```
    std::string resp;
```

```
    if (!SendCommand(sock, cmd, resp)) break;
```

```
    std::cout << "\n== Server response ==\n" << resp << "=====\n";
```

```
    if (cmd == "*Quit") break;
```

```
}
```